1 RC4 Stream Cipher Implementation in C

```c
/*Rupayan Mandal, CRS2513*/
/*TO COMPILE-> gcc rc4.c -o test -o1*/

#include <stdio.h>
#include <stdint.h>
#include <stdlib.h>
#include <string.h>

#include <x86intrin.h>

#define MAX_MSG_LEN 1000
#define STEPS 100



/* ---- cycle counting
   -------------------------------------------------*/

static inline unsigned long long start_tsc(void)
{
    _mm_lfence();//It prevents the CPU out-of-order
       execution engine from speculatively executing
       instructions across the timing boundary, ensuring
       precise timing measurements.
    return __rdtsc();
}
static inline unsigned long long stop_tsc(void)
{
    unsigned int aux;
    unsigned long long tsc = __rdtscp(&aux);
    _mm_lfence();
    return tsc;
}

void init_S(unsigned char* S)
{
    for (int i = 0; i < 256; i++)
        S[i] = i;
}
/*KEY SCHEDULING ALGORITHM (KSA)*/
double KSA(unsigned char* S,unsigned char key[], size_t
```

```
    key_len )
{
     int i=0;
     int j=0;
     unsigned char temp;
     unsigned char K[256];
     for(int i=0;i<256;i++)
     {
     K[i]=key[i%key_len];
     }
     unsigned long long start_cycle=start_tsc();
     for(i=0;i<256;i++)
     {
          j=(j+S[i]+K[i]) &0XFF;     // &0XFF -> mod 256

          temp=S[i];
          S[i]=S[j];
          S[j]=temp;
          /*used 3 assignments instead of 3XORS as it is
             faster
          in modern cpu 3 dependant XORs prevent
             parallelism */
     }
     unsigned long long end_cycle=stop_tsc();
     return (end_cycle-start_cycle);
}

double PRGA(unsigned char* S,unsigned char* keystream,
    size_t msglen)
{
     int i=0;
     int j=0;
     int k;
     unsigned char temp;

     unsigned long long start_cycle=start_tsc();
     for (k=0;k<msglen;k++)
     {
          i=(i+1) &0XFF;
          j=(j+S[i]) &0XFF;
          temp=S[i];
          S[i]=S[j];
          S[j]=temp;
```

```
77          keystream[k]=S[(S[i]+S[j]) &0XFF];
78          //printf("\n%d 0x%X",k, keystream[k]);
79      }
80      unsigned long long end_cycle=stop_tsc();
81
82      return (end_cycle-start_cycle)/(double)msglen;
83
84  }
85
86  int gen_msg(unsigned char*msg,size_t msg_len)
87  {
88      for (int i = 0; i <msg_len; i++)
89      {
90          msg[i] = i & 0xFF;
91      }
92  }
93
94  int rc4(unsigned char* S, unsigned char key[],size_t
        keylen,unsigned char* msg, size_t msg_len)
95  {
96    unsigned long long start_cycle, end_cycle;
97
98      double total_ksa_cycles, cycles_per_byte_prga;
99
100
101     size_t msglen = msg_len;
102     printf("\nKEYLEN %zu",keylen);
103
104     unsigned char* keystream=(unsigned char*)malloc(
            msglen*sizeof(unsigned char));
105     unsigned char* ciphertext=(unsigned char*)malloc(
            msglen*sizeof(unsigned char));
106     unsigned char* plaintext=(unsigned char*)malloc(
            msglen*sizeof(unsigned char) +1);
107
108     if (!keystream || !ciphertext || !plaintext) {
109     fprintf(stderr, "Memory allocation failed\n");
110     return 1;}
111
112     int i;
113     for (i=0;i<256;i++)
114     {
115         S[i]=i;
```

```
116     }
117
118     unsigned int temp;
119     unsigned int j=0;
120
121     /*KEY SCHEDULING ALGORITHM (KSA)*/
122     KSA(S,key,keylen);  // warmup
123     init_S(S);
124     total_ksa_cycles=KSA(S,key,keylen); //for
           benchmarking
125
126     /*PSEUDO-RANDOM GENERATION ALGORITHM (PRGA)*/
127     unsigned char* tempS=(unsigned char*)malloc(256*
           sizeof(unsigned char));
128     memcpy(tempS,S,256);
129     PRGA(S,keystream,msglen); //warmup
130     PRGA(S,keystream,msglen);
131     memcpy(S,tempS,256);
132     cycles_per_byte_prga=PRGA(S,keystream,msglen); //for
           benchmarking
133     free(tempS);
134     int k;
135     /* for(k=0;k<msglen;k++)
136     {
137         printf("\n%d 0x%X",k,keystream[k]);
138     } */
139
140     for(k=0;k<msglen;k++)
141     {
142         ciphertext[k]=keystream[k]^msg[k];
143     }
144
145     printf("\nNumber of Encrypted bytes: %d",k);
146
147
148
149     free(keystream);
150     free(ciphertext);
151     free(plaintext);
152     printf("\n
           ==================================================\
           n");
153     printf("KSA : Total cycles      :%.2f\n", (
```

```
        total_ksa_cycles));
154     printf("KSA : cycles/byte       :%.2f CPB\n", (
        total_ksa_cycles/256.0));
155     printf("PRGA: cycles/byte       :%.2f CPB\n",
        cycles_per_byte_prga);
156     printf("
        ====================================================\
        n");
157 }
158 int main()
159 {
160
161
162     /*INItTIALISATION*/
163     unsigned char* S=(unsigned char*)malloc(256*sizeof(
        unsigned char));
164
165     unsigned char key[]={
166     0x01, 0x02, 0x03, 0x04,
167     0x05, 0x06, 0x07};
168
169     size_t keylen=sizeof(key);
170
171     size_t msg_len=1000;
172     for(msg_len=100;msg_len<MAX_MSG_LEN;msg_len=msg_len+
        STEPS)
173     {
174         unsigned char* msg =malloc(msg_len);
175         gen_msg(msg,msg_len);
176         rc4(S,key,keylen,msg,msg_len);
177         free(msg);
178     }
179
180
181
182     return 0;
183 }
```

2 Output: in Windows system

Encrypted Bytes	KSA Total Cycles	KSA Cycles/Byte	PRGA Cycles/Byte
100	4608.00	18.00	19.46
200	3762.00	14.70	17.16
300	3752.00	14.66	15.99
400	3520.00	13.75	15.73
500	3936.00	15.38	15.34
600	3956.00	15.45	15.35
700	3756.00	14.67	15.68
800	3724.00	14.55	15.78
900	2886.00	11.27	15.84

Table 1: Cycle counts for RC4 KSA and PRGA with key length 7 bytes.

3 Output: in LINUX system (UBUNTU)

Encrypted Bytes	KSA Total Cycles	KSA Cycles/Byte	PRGA Cycles/Byte
100	1092.00	4.27	7.56
200	2142.00	8.37	9.45
300	2226.00	8.70	10.92
400	2100.00	8.20	8.87
500	2331.00	9.11	9.83
600	2184.00	8.53	8.86
700	2163.00	8.45	9.63
800	2226.00	8.70	8.66
900	2226.00	8.70	8.75

Table 2: Cycle counts for RC4 KSA and PRGA with key length 7 bytes

NOTE

in both cases compiled with " `gcc -O1 -march=native`" but since gcc is optimised for linux systems so it's giving a higher order counts in the windows system.

4 Analysis of RC4 Key Scheduling Algorithm (KSA)

4.1 Algorithm Recap

The KSA initializes the permutation array $S[256]$ using the key. For each iteration i ($0 \leq i < 256$):

$$j = (j + S[i] + K[i]) \bmod 256$$

$$\text{swap}(S[i], S[j])$$

Operations per Iteration

Each iteration involves:

- 2 integer additions
- 1 bitwise AND (for modulo 256)
- 3 assignments (for the swap)

Thus, the total primitive operations per iteration are:

$$2 + 1 + 3 = 6$$

Total Operations

Since the loop runs for 256 iterations:

$$256 \times 6 = 1536 \quad \text{primitive operations}$$

Cycle Cost Estimation

Approximate cycle costs on a modern CPU:

- Integer addition: ≈ 1 cycle
- Bitwise AND: ≈ 1 cycle
- Assignment (load/store): ≈ 1–2 cycles

Therefore, per iteration:

$$2 \times 1 + 1 \times 1 + 3 \times (1 \text{ to } 2) \approx 6 \text{ to } 9 \quad \text{cycles}$$

Total Cycle Estimate

$$256 \times (6 \text{ to } 9) \approx 1536 \text{ to } 2304 \quad \text{cycles}$$

5 Analysis of RC4 Pseudo-Random Generation Algorithm (PRGA)

Algorithm Recap

For each output byte, PRGA performs the following steps:

$$i = (i + 1) \bmod 256$$

$$j = (j + S[i]) \bmod 256$$

$$\text{swap}(S[i], S[j])$$

$$\text{keystream}[k] = S[(S[i] + S[j]) \bmod 256]$$

Operations per Byte

Each iteration involves:

- 2 integer additions
- 2 modulo 256 operations (implemented as bitwise AND)
- 3 assignments (for the swap)
- 1 array access and assignment (keystream output)

Thus, the total primitive operations per byte are:

$$2 + 2 + 3 + 1 = 8$$

Cycle Cost Estimation

Approximate cycle costs on a modern CPU:

- Integer addition: ≈ 1 cycle
- Bitwise AND: ≈ 1 cycle
- Assignment (load/store): ≈ 1–2 cycles
- Array access: ≈ 1–2 cycles (assuming cache hit)

Therefore, per byte:

$$2 \times 1 + 2 \times 1 + 3 \times (1 \text{ to } 2) + 1 \times (1 \text{ to } 2) \approx 8 \text{ to } 11 \quad \text{cycles}$$